Cake Pattern

Season: 7 · Advanced FP · Category: Functional · Level: Advanced

Intent

- Mixin components via interfaces (self-type emulation).

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Layered cake of UserRepo + Email + UserService.

When to Use

- Serious FP architecture in Java.
- The design force named in Intent shows up more than once.

When Not to Use

- CRUD apps that don't need effect algebras.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.cake;

/**
 * Cake pattern (Scala) emulated with Java interfaces + default methods.
 * Self-type: a component declares what it needs via abstract getters;
 * the "cake" assembles mixins into one object.
 */
public final class CakePattern {
    private CakePattern() {}

    public interface UserRepoComponent {
        UserRepo userRepo();
    }

    public interface UserRepo {
        String findEmail(String id);
    }

    public interface EmailServiceComponent {
        EmailService emailService();
    }

    public interface EmailService {
        String send(String to, String body);
    }

    public interface UserServiceComponent extends UserRepoComponent, EmailServiceComponent {
        default UserService userService() {
            return id -> {
                var email = userRepo().findEmail(id);
                return emailService().send(email, "welcome " + id);
            };
        }
```

Classic Implementation (2)

```
    }

    public interface UserService {
        String welcome(String id);
    }

    /** The cake — wires live implementations. */
    public static final class App implements UserServiceComponent {
        public UserRepo userRepo() { return id -> id + "@example.com"; }
        public EmailService emailService() { return (to, body) -> "sent:" + to + ":" + body; }
    }
}
```

Modern Java 25 / Functional

- Witness types emulate higher-kinded types.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/cake/
```

Pitfalls

- Encodings are educational — production may use libraries.
- Tagless Final needs Kind encoding without HKTs.

Interview Q&A

- Q: When to pick Cake Pattern? A: When you need testable effect boundaries or pure state threading.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Mixin components via interfaces (self-type emulation).
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Layered cake of UserRepo + Email + UserService. [0:40] Intent — State the design force in one senten...